

Documentação Técnica - Fase IV

Alunos:

- Felipe Portes Antunes
- Lucas Teixeira Reis
- Thayná Andrade Caldeira Antunes

Professor: Walisson Ferreira de Carvalho

Link do Projeto: [Checklist App](#)

Link do vídeo desta etapa: [Exemplo de funcionamento do Backup](#)

Relatório Técnico e Formulário

Este documento descreve as implementações realizadas na Fase IV do projeto Checklist App, focando na aplicação de algoritmos de compressão para backup dos dados.

1. Qual foi a taxa de compressão obtida com o algoritmo de Huffman?

- **a. Tamanho do arquivo original:** 2.535 bytes
- **b. Tamanho do arquivo comprimido:** 1.636 bytes
- **c. Cálculo da taxa:** $(1 - 1636 / 2535) * 100 = 35,46\%$
- **d. Interpretação do resultado:** O algoritmo de Huffman reduziu o tamanho total dos dados em aproximadamente 35%. Esta taxa é considerada moderada e ocorre porque Huffman é mais eficiente em arquivos com alta redundância de caracteres, e nossos arquivos binários de banco de dados possuem uma distribuição de bytes relativamente variada.

2. Qual foi a taxa de compressão obtida com o algoritmo de LZW?

- **a. Tamanho do arquivo original:** 2.535 bytes
- **b. Tamanho do arquivo comprimido:** 976 bytes
- **c. Cálculo da taxa:** $(1 - 976 / 2535) * 100 = 61,50\%$
- **d. Interpretação do resultado:** O algoritmo LZW obteve uma performance superior, reduzindo o tamanho dos arquivos em mais de 60%. Isso ocorre porque o LZW é excelente para identificar e codificar sequências repetidas de bytes, o que é comum em nossos arquivos de índice e registros que possuem estruturas fixas e metadados repetitivos.

3. Quais dificuldades surgiram ao implementar Huffman e LZW e como você resolveu?

Dificuldades Huffman:

1. **Persistência da Árvore:** Para que a descompressão funcione, o descompressor precisa da mesma árvore de Huffman usada na compressão.
 - **Solução:** Implementei um cabeçalho no arquivo comprimido que armazena a tabela de frequências de cada byte. Assim, o descompressor reconstrói a árvore de forma idêntica antes de iniciar a leitura dos dados.
2. **Manipulação de Bits:** Java trabalha nativamente com bytes, mas Huffman gera códigos de comprimentos variáveis de bits.
 - **Solução:** Utilizei um `StringBuilder` para acumular a sequência de bits como strings '0' e '1' e depois converti essa string em um array de bytes, tratando o "padding" no último byte através da gravação da quantidade exata de bits válidos no cabeçalho.

Dificuldades LZW:

1. **Gerenciamento do Dicionário:** O dicionário pode crescer indefinidamente, consumindo muita memória.
 - **Solução:** Limitei o tamanho do dicionário a 4096 entradas (12 bits por código). Uma vez atingido esse limite, o algoritmo para de adicionar novas sequências e continua a compressão usando apenas os padrões já mapeados.
2. **Empacotamento de Códigos de 12 bits:** Gravar códigos de 12 bits em um sistema de arquivos baseado em bytes (8 bits) exige deslocamento de bits.
 - **Solução:** Implementei um sistema de "bit buffering" que acumula os bits dos códigos de 12 bits e descarrega bytes completos (8 bits) conforme o buffer enche, garantindo o uso eficiente do espaço.

Dificuldade Geral (Backup de Múltiplos Arquivos):

- **Compactação em Arquivo Único:** O requisito exigia um único arquivo compactado contendo todos os arquivos de dados do sistema.
 - **Solução:** Criei um formato de “pre-bundle” no BackupManager . Antes de comprimir, todos os arquivos do diretório ./dados são lidos e concatenados em um único array de bytes seguindo a estrutura: [Nome do Arquivo (UTF)][Tamanho (Long)][Conteúdo (Bytes)] . Esse array gigante é então enviado para o algoritmo de compressão. Na descompressão, o processo inverso restaura cada arquivo em seu local original.
-